# Should You Judge A Movie By Its Poster?

Adi Agarwal, Anika Misra, Alex Roesler

December 6, 2024

**Abstract**

The objective of this project is to use a Convolutional Neural Network (CNN) to predict a movie's revenue based on its poster. This can be applied by movie producers to determine the ideal movie poster for generating the most revenue. In the following report, we modify a pretrained CNN, Resnet-50, to perform the task. We broadly explain how CNNs work, describe our dataset and pre-processing steps, explain our model, and discuss future improvements. The link to our GitHub repository can be found at `https://github.com/revel90024/156_project/`.

## 1 Introduction

While exploring datasets on Hugging Face, we discovered a comprehensive dataset [11] of movie posters that includes additional features such as budget, revenue, popularity, genre, and title. This prompted us to investigate whether a movie's revenue could be predicted solely from the image of its poster. CNNs are particularly well-suited for this task due to their exceptional ability to extract and classify features from images [12]. Recognizing that it is inherently difficult for humans to accurately forecast a movie's revenue based merely on its poster, we hypothesize that an effective CNN model can identify hidden features correlated with higher or lower revenues. To maximize our model's accuracy, our objective is to determine the optimal CNN architecture that can accurately discern meaningful features from movie posters, integrate these features into an ideal configuration of hidden layers with the appropriate number of neurons, and incorporate dropout layers to prevent overfitting.

**General Approach:** We modified a pre-existing classic architecture, ResNet-50, and experimented with different parameters, such as hidden and dropout layers, until we found the most accurate revenue predictions. We analyzed the accuracy of the results by checking if the model's revenue prediction was within a certain percentage of the actual revenue value.

## 2 Background

Basic CNNs are composed of three types of layers: **convolutional layers**, **pooling layers**, and **fully-connected layers** [12]. A visual example of a simple CNN with one of each type of these layers is provided in Figure 1.

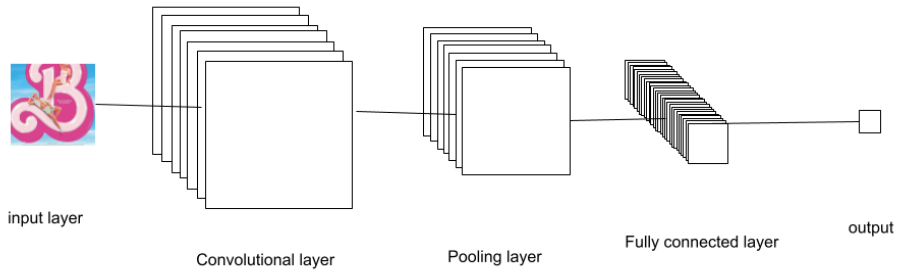


Figure 1: A simple CNN diagram.

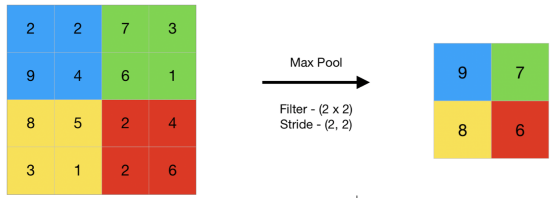


Figure 2: An example of max-pooling. Source [8]

### 2.1 Input Layer

The input layer contains the pixel values of the image. These pixel values are in the form of height x width x depth [12]. The depth might be a feature such as the three-dimensional color channel (red, green, blue). In our model, due to the input requirements of ResNet-50, our images are resized to height and width of 224 x 224 pixels [3].

### 2.2 Convolutional Layer

The convolutional layer applies a set of kernels, or a set of connection weights, to the local regions of the previous layer. A complete convolutional layer is composed of several such kernels [7]. The equation for a convolution utilizing a 2-dimensional kernel [4] is shown in Equation 1. The output of each convolution is called a **feature map** [4]. An activation function, such as ReLu 2, is also applied in order to capture a non-linear pattern in the data.

$$S(i,j) = (K * I)(i,j) = \sum_m \sum_n I(i-m, j-n)K(m,n) \tag{1}$$

### 2.3 Pooling Layer

The pooling layer reduces the dimensionality of the previous layer by operating over each feature map independently. One example of pooling is max-pooling, which applies a 2x2 kernel with a stride of 2 to find the associated maximum values in each feature map. This operation reduces dimensionality and extracts the important information [12]. A visualization of max-pooling is provided in Figure 2. The output of each convolution is called a feature map [4]. An activation function, such as ReLU 2, is also applied in order to capture a non-linear pattern in the data [12].

$$\text{ReLU}(x) = \max(0, x) \tag{2}$$

### 2.4 Fully-connected Layer

Finally, the fully-connected layer produces a one-dimensional classification score through matrix multiplication of its input, often utilizing a non-linear activation function such as the Softmax function [6] as shown in Equation 3, which allows outputs to be interpreted as probabilities. Another example of an activation function is ReLU as shown in Equation 2, which was used in our fully-connected layers. The input for the fully-connected layer need not be a feature map, as multiple fully-connected layers can be stacked in a row.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{3}$$

## 3 Dataset

The dataset [11] we are utilizing was downloaded from HuggingFace and does not present any major ethical concerns, as all data was acquired with consent and consists of public information such as movie posters and related details. It encompasses a wide range of genres, topics, and characters, thereby avoiding explicit bias; any implicit bias present reflects those inherent in the film industry, given that our data serves as a comprehensive representation of movies produced within it. Although the dataset was created by a Swedish individual, potentially introducing some bias, the vast majority of movies are still presented in English. Initially, the dataset contained 95.3k entries with numerous duplications. By removing all movies with

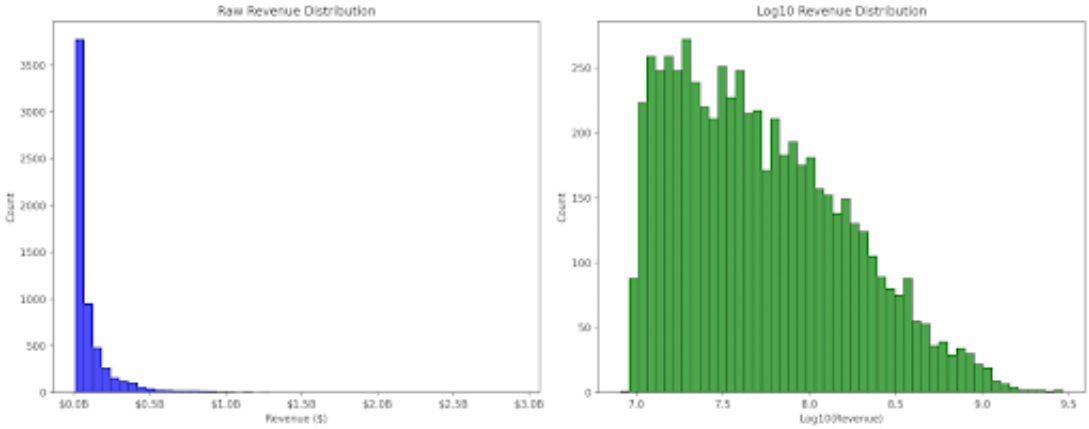


Figure 3: Log scale normalization.

revenue under $10 million—a reasonable threshold considering that most widely viewed movies generate at least this amount—we reduced the dataset to approximately 14k movies. After deduplication, we were left with 6,137 unique movies, each with revenues exceeding $10 million. The images were already available in torchvision tensor format, eliminating the need for conversion. Given that the revenue values exhibited a left-skewed distribution, we applied a log transformation with base 10 to achieve a distribution more similar to a normal distribution. This transformation enhanced the accuracy of our regression-based model by reducing bias toward movies with lower revenues. For exploratory analysis, we visualized the revenue distribution before and after the log scaling, demonstrating that the log base 10 scaling effectively normalized the dataset, thereby improving its suitability for regression models [2].

# 4 Model

We are employing the ResNet architecture, a seminal model renowned for integrating pooling layers within its convolutional layers. Specifically, we have adapted the structure of ResNet-50, a residual network composed of 50 layers. ResNet-50 enhances the original 34-layer residual network in the above picture, by substituting each two-layer block with a three-layer bottleneck block. This three-layer bottleneck block comprises three consecutive convolutional layers meticulously designed to reduce computational complexity while maintaining robust feature extraction capabilities. The first and third layers utilize 1x1 convolutions to decrease and then restore the dimensionality, respectively. This design allows the second layer, a 3x3 convolution dedicated to feature extraction, to operate on feature maps with reduced input and output dimensions. By compressing the feature maps in the intermediate layer, the primary convolution with a larger kernel size requires significantly fewer parameters and computations, thereby increasing efficiency and enabling the network to scale effectively for deeper architectures [6, 3]. We modified the output layer of ResNet-50, where instead of using the fully connected layer with 1000 neurons as was used when classifying the ImageNet dataset with 1000 classes, we used three hidden layers. ResNet-50 outputs a 2048-dimensional feature vector, and our three hidden layers are dense layers with 1024, 512, and 1024 neurons respectively. For all three hidden layers we used ReLU activation functions, and used dropout rates of 0.2, 0.1, and 0.2 respectively. Our output layer has one neuron, as we wanted our model to output a predicted revenue for each inputted movie poster. Since our inputted data was on a log scale, we must raise 10 to the power of our output value to revert back to the predicted revenue in dollars. We did not use a softmax activation function on our output layer because we want a raw revenue prediction, not a probability.

When training our model, we used 30 epochs with early stopping after 7 non-improving epochs. AdamW optimizer was used, which "is a stochastic gradient descent method that is based on adaptive estimation of first-order and second-order moments [9]." Instead of the Adam optimizer, which incorporates weight decay as part of the gradient update, AdamW decouples the weight decay from the gradient computation. In AdamW, when weight decay does not interfere with the optimization process, we see a more stable training and better generalization [9]. Since this is fundamentally a regression problem, we used MSE loss instead of cross-entropy loss, which is preferred for classification [5]. For evaluation metrics, we used a percentage of predictions that are within 25% of the actual revenue values, as well as the percentage

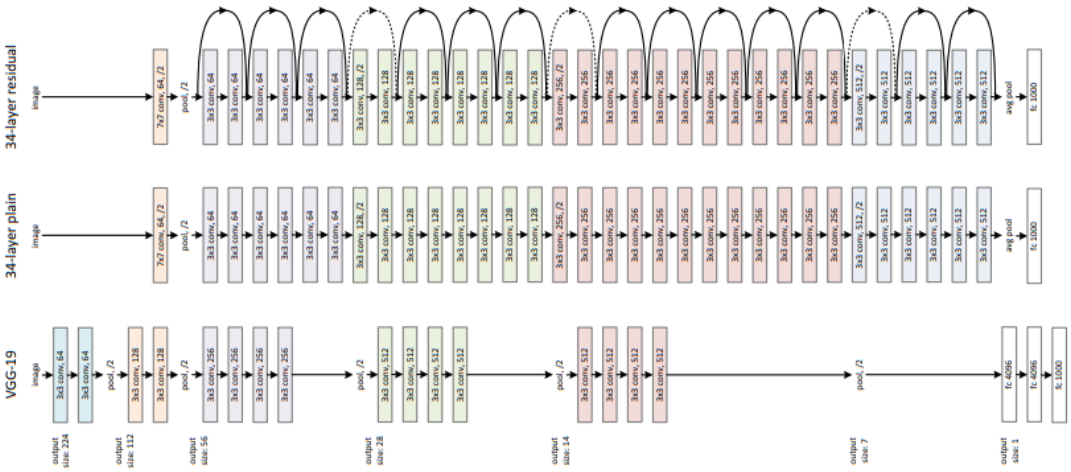


Figure 4: Example network architectures for ImageNet. Source [6]

within 50% of the actual revenue values. These show us how often the model gets substantially close to the actual revenue, and how often the model gets somewhat close to the actual revenue. We also used accuracy, precision, recall, and F1 score to show us how effective our model is at predicting whether a movie's revenue is above or below the median. Note also the data was split into 80% training, 10% validation, 10% testing. There was no crossover within these sets for our test results.

# 5 Results

**Best Epoch Metric**

| Metric | Value |
|---|---|
| Validation Loss | 0.22742842882871628 |

Table 1: The best epoch had a validation loss of approximately 0.227.

**Parameters**

| Parameter | Value |
|---|---|
| Learning Rate | 0.0002 |
| Weight Decay | 0.0001 |
| Epochs | 30 |
| Patience | 7 |
| L1 Lambda | 0 |
| Clip Gradient | Null |
| Batch Size | 32 |
| Hidden Sizes | [1024, 512, 1024] |
| Dropout Rates | [0.2, 0.1, 0.2] |

Table 2: Table showcasing the parameters of the model with the best performance, including learning rate, weight decay, number of epochs, etc.

**Test Metrics**

| Metric | Value |
|---|---|
| Test Loss | 0.23986979946494102 |
| Prediction Within 25% of Actual Value | 17.79935275080906% |
| Prediction Within 50% of Actual Value | 35.59870550161812% |
| Accuracy (above/below median revenue) | 57.76699029126213% |
| Precision | 57.84313725490197% |
| Recall | 57.28155339805825% |
| F1 Score | 57.5609756097561% |

Table 3: Table showcasing test metrics used for our model, as well as their final values.

## Analysis and Insights

There are certainly some improvements that could be made. A bucketing approach to the output instead of a continuous number output, the resizing of images to $224 \times 224$ as ResNet needed may have convoluted some key information, and better distribution of movie choice (e.g., English only, modern movies, etc.) could possibly improve performance. The dataset is exponentially distributed, and there are possibly better architectures to handle this than what we chose. Furthermore, we did not have time to test a variety of hyperparameters, although increasing the learning rate and decreasing regularization did seem to improve results slightly.

In the end, intuitively, there is likely little discernible correlation between how well a movie does and its poster, as the process of poster-making is quite standardized. The model was likely finding correlations of attributes (genres, studios, style) and basing its predictions off those. For the model to be guessing as well as it did with 17.8% of movies within a 25% error, 35.6% within 50% (note: exactly double), and 57.8% in the correct half above or below the median is surprising.

## A Fun Insightful Side Quest

We wanted to test our model on newer movies that our dataset (updated in 2023) did not contain, so we compiled our own test dataset [1] to see how it would handle the following random, biased (not statistically significant) dataset. This is not rigorous testing but provides informative insights into our process.

### Model 1: Our Final Model

| Title | Predicted | Actual | Error |
|---|---|---|---|
| Deadpool & Wolverine | \$43.0M | \$1.3B | 96.8% |
| Kung Fu Panda 4 | \$45.7M | \$547.6M | 91.6% |
| Alien: Romulus | \$36.9M | \$350.9M | 89.5% |
| Beetlejuice | \$25.8M | \$294.1M | 91.2% |
| A Quiet Place: Day One | \$42.5M | \$261.8M | 83.8% |
| Bob Marley: One Love | \$41.0M | \$180.8M | 77.3% |
| Civil War | \$28.7M | \$126.2M | 77.3% |
| Reagan | \$30.4M | \$30.1M | 0.9% |
| Ministry of Ungentlemanly Warfare | \$38.7M | \$27.3M | 41.6% |

Table 4: Comparison of predicted revenues vs actual revenues on the newer movie test set. Predicted values were calculated using our previous model and parameters.

### Model 2: Trained on Revenue > \$50M

| Title | Predicted | Actual | Error |
|---|---|---|---|
| Deadpool & Wolverine | \$73.5M | \$1.3B | 94.5% |
| Kung Fu Panda 4 | \$133.4M | \$547.6M | 75.6% |
| Alien: Romulus | \$96.5M | \$350.9M | 72.5% |
| Beetlejuice | \$120.3M | \$294.1M | 59.1% |
| A Quiet Place: Day One | \$81.8M | \$261.8M | 68.8% |
| Bob Marley: One Love | \$43.9M | \$180.8M | 75.7% |
| Civil War | \$59.0M | \$126.2M | 53.3% |
| Reagan | \$61.7M | \$30.1M | 105.1% |
| Ministry of Ungentlemanly Warfare | \$64.6M | \$27.3M | 136.4% |

Table 5: Comparison of predicted revenues vs actual revenues on the newer movie test set. Predicted values were calculated using a model with our parameters, but trained on a dataset containing only movies that had a revenue > \$50M. We can see that the results are better than in Table 4.

### Model 3: Trained on small Dataset, Log10 Scaling

| Title | Predicted | Actual | Error |
|---|---|---|---|
| Deadpool & Wolverine | \$109.6M | \$1.3B | 91.8% |
| Kung Fu Panda 4 | \$131.7M | \$547.6M | 75.9% |
| Alien: Romulus | \$56.2M | \$350.9M | 84.0% |
| Beetlejuice | \$69.9M | \$294.1M | 76.2% |
| A Quiet Place: Day One | \$67.8M | \$261.8M | 74.1% |
| Bob Marley: One Love | \$95.5M | \$180.8M | 47.2% |
| Civil War | \$38.7M | \$126.2M | 69.3% |
| Reagan | \$64.2M | \$30.1M | 113.5% |
| Ministry of Ungentlemanly Warfare | \$74.4M | \$27.3M | 172.5% |

Table 6: Comparison of predicted revenues vs actual revenues on the newer movie test set. Predicted values were calculated using a model with the same parameters as our previous one, but trained on a smaller dataset.

**Model 3: Trained on small Dataset, Ln Scaling**

| Title | Predicted | Actual | Error |
|---|---|---|---|
| Deadpool & Wolverine | $265.6M | $1.3B | 80.1% |
| Kung Fu Panda 4 | $98.0M | $547.6M | 82.1% |
| Alien: Romulus | $65.5M | $350.9M | 81.3% |
| Beetlejuice | $262.8M | $294.1M | 10.6% |
| A Quiet Place: Day One | $342.8M | $261.8M | 31.0% |
| Bob Marley: One Love | $48.1M | $180.8M | 73.4% |
| Civil War | $52.7M | $126.2M | 58.2% |
| Reagan | $326.4M | $30.1M | 984.3% |
| Ministry of Ungentlemanly Warfare | $118.8M | $27.3M | 335.0% |

Table 7: Comparison of predicted revenues vs actual revenues on the newer movie test set. Predicted values were calculated using a model with the same parameters as our previous one, but trained on a smaller dataset, and normalized using exponential scaling.

We observed that models trained on a smaller dataset [10] performed surprisingly better for this specific test set, possibly due to better representation of high-revenue movies, as seen in Tables 6 and 7. However, the larger dataset that we used in our training yielded better overall validation and test loss, suggesting better generalization. Despite improvements, there remains a challenge in predicting revenue from posters alone due to the standardized process of poster design. As seen in Tables 6 and 7, using different bases when log scaling will change the rate at which our distribution is spread out, and lead to different performances, not definitively better or worse for our last two tables above.

# 6 Conclusion

Our goal with this project was to create a model that could accurately predict a movie's revenue with just its poster. Considering the extremely large range of revenues we had in our dataset, even above $10 million in revenue, our model did better than expected with direct predictions. Our classification metrics, which evaluated the model's ability to predict whether revenue is above or below the median revenue, were somewhat underwhelming, as we barely broke 50% for these metrics.

# 7 Contributions

- Adi: Did the introduction and the dataset description. Contributed to the results section in the report. Preprocessed the data, tuned the training process (epochs, loss function, optimizer), and trained the model.
- Alex: Did the title, author list, abstract, and model description. Contributed to the conclusion and potential improvements sections in the report. Selected the ResNet-50 model and modified it to work with regression instead of classification. Helped with tuning the fully-connected layers and dropout layers of the model.
- Anika: Did the background, contributions, acknowledgements, and references. Contributed to the results, conclusion, and potential improvements sections in the report. Helped with tuning the fully-connected layers and dropout layers of the model. Calculated the metrics for the model.

# 8 Acknowledgments

We did not receive any external help, except from 3Blue1Brown on YouTube to help us understand convolutions in CNNs.

# 9 Potential Improvements

- We believe our model could have been improved by incorporating budgets, which were included in the dataset. Budgets could provide valuable context to help the model assess the expected magnitude of revenue. To achieve this, we would have employed an ensemble model that first passed the data through the same CNN used to extract meaningful features from the movie posters. These extracted features, combined with budgets in a neural network (potentially a recurrent neural network), could then predict the revenues generated by the movies.

- Given more time, we would have investigated the impact of further transforming our dataset to approximate a normal distribution. Since our dataset was very large, training the model was computationally expensive. Approximating a normal distribution could potentially enhance model performance, as regression-based models generally perform better with data that aligns closely with normality. Furthermore, we could have explored models specifically designed to handle exponentially distributed data, as our dataset was inherently closer to an exponential distribution than a normal one.

# References


[1] Adi Agarwal. movie-data, 2024. Accessed on Dec 5, 2024. URL: https://github.com/revel90024/156_project/blob/main/fun_posters/movie-data.json.

[2] Reine Carreira. Handling skewed data. *Medium*, 2021. Accessed on Dec 5, 2024. URL: https://reinec.medium.com/my-notes-handling-skewed-data-5984de303725.

[3] The Linux Foundation. Pytorch, 2017. Accessed on Dec 5, 2024. URL: https://pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html.

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Accessed on Dec 5, 2024. URL: http://www.deeplearningbook.org.

[5] Trung Hatodi. A comparison between mse, cross-entropy, and hinge loss, 2023. Accessed on Dec 5, 2024. URL: https://medium.com/@hatodi0945/a-comparison-between-mse-cross-entropy-and-hinge-loss-4d4fe63cca12#:~:text=The%20MSE%20loss%20is%20therefore,several%20iterations%20of%20model%20training.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. Accessed on Dec 5, 2024. URL: http://arxiv.org/abs/1512.03385, arXiv:1512.03385.

[7] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. Accessed on Dec 5, 2024. doi:10.1109/5.726791.

[8] Sanchhaya Education Private Limited. Geeksforgeeks, 2024. Accessed on Dec 5, 2024. URL: https://www.geeksforgeeks.org/cnn-introduction-to-pooling-layer/.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. Accessed on Dec 5, 2024. URL: https://arxiv.org/abs/1711.05101, arXiv:1711.05101.

[10] Tim Olsén. Movie posters, 2023. Accessed on Dec 2, 2024. URL: https://huggingface.co/datasets/skvarre/movie-posters.

[11] Tim Olsén. Movie posters 100k, 2023. Accessed on Dec 2, 2024. URL: https://huggingface.co/datasets/skvarre/movie_posters-100k/viewer.

[12] Keiron O'Shea and Ryan Nash. An Introduction to Convolutional Neural Networks. *arXiv e-prints*, page arXiv:1511.08458, November 2015. Accessed on Dec 5, 2024. arXiv:1511.08458, doi:10.48550/arXiv.1511.08458.